

模 型 评 估

时间：2025年5月

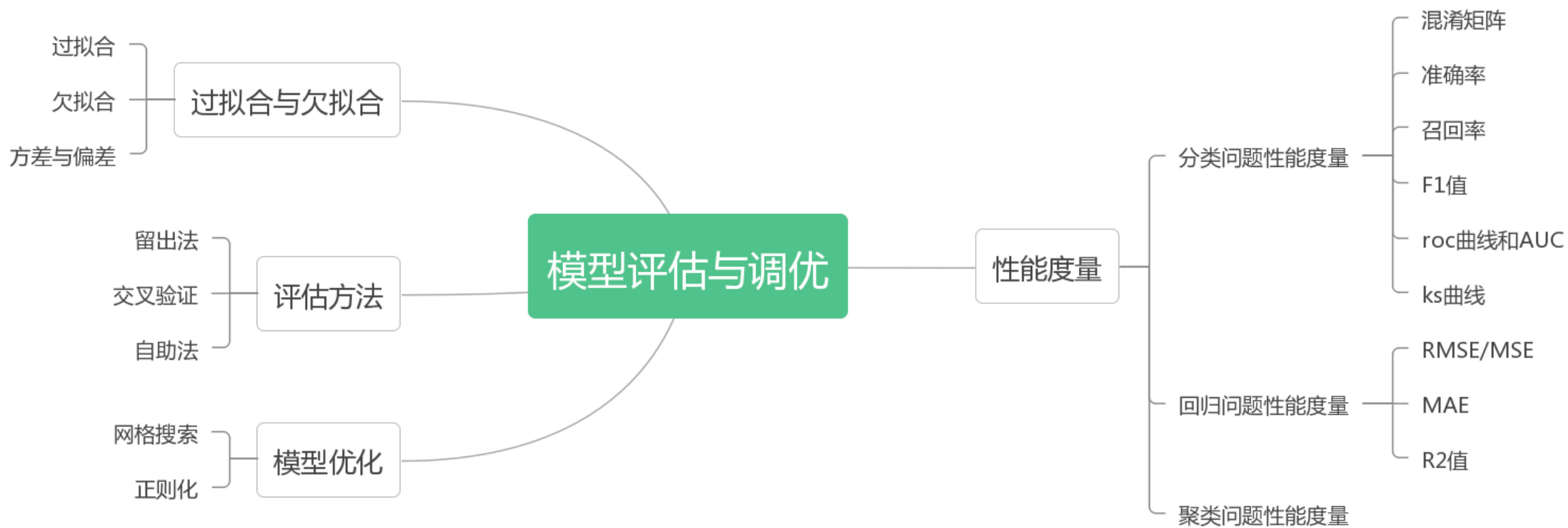
TRANSWARP
星 环 科 技



目录

CONTENTS

- 1 过拟合和欠拟合
- 2 评估方法
- 3 性能度量





1 chapter

过拟合和欠拟合

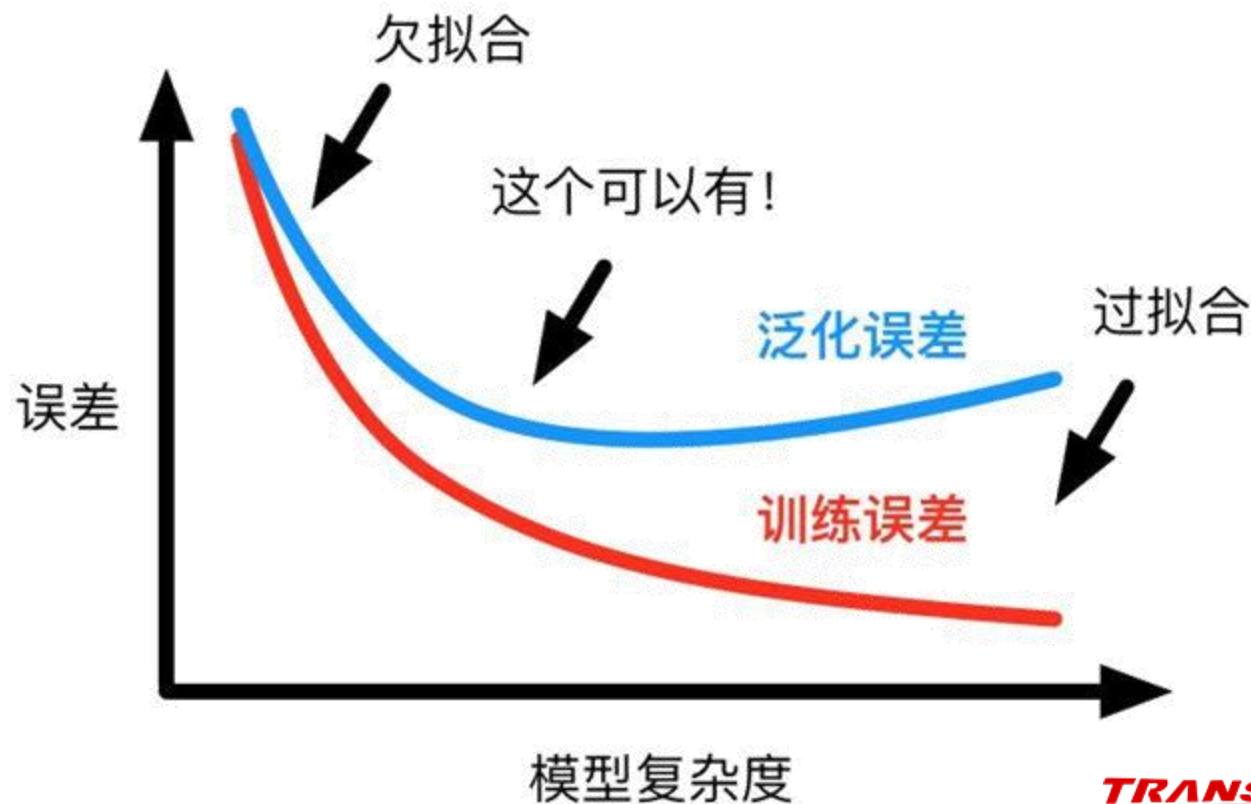
训练误差和泛化误差

训练误差：模型在**训练集（历史数据）**上的错误率（做**练习**的错误率）

泛化误差：模型在**未知数据**上的错误率（**考试**的错误率）

过拟合：训练误差低，泛化误差高

欠拟合：训练误差和泛化误差都高

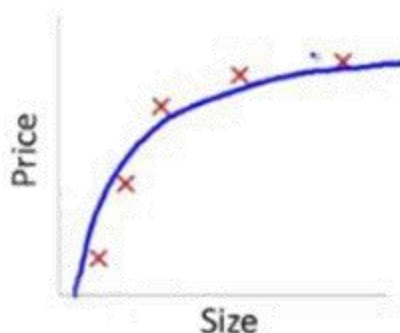


训练误差和泛化误差



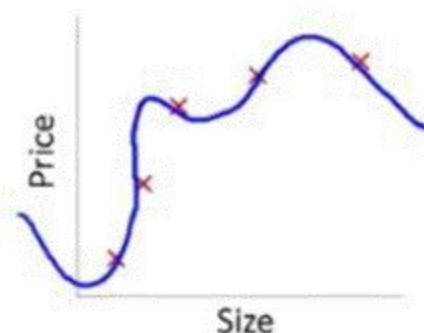
$$\theta_0 + \theta_1 x$$

High bias
(underfit)



$$\theta_0 + \theta_1 x + \theta_2 x^2$$

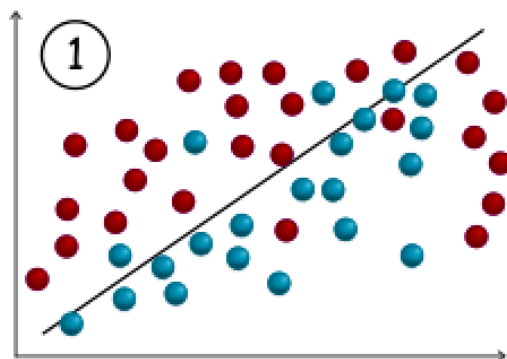
"Just right"



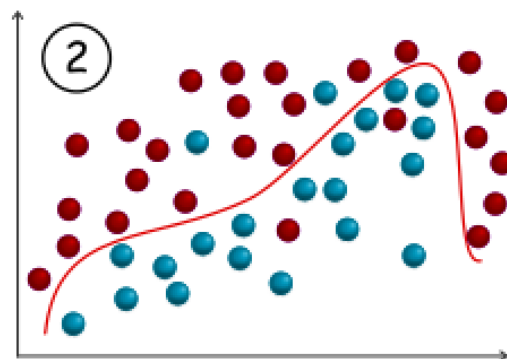
$$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$$

High variance
(overfit)

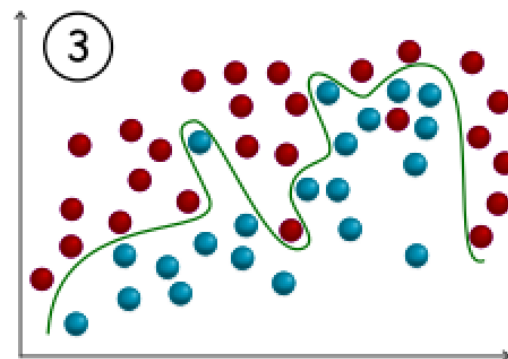
underfitting



good fit



overfitting



方差和偏差 泛化误差 (Generalization Error) $\approx \text{Bias}^2 + \text{Variance} + \text{Noise}$

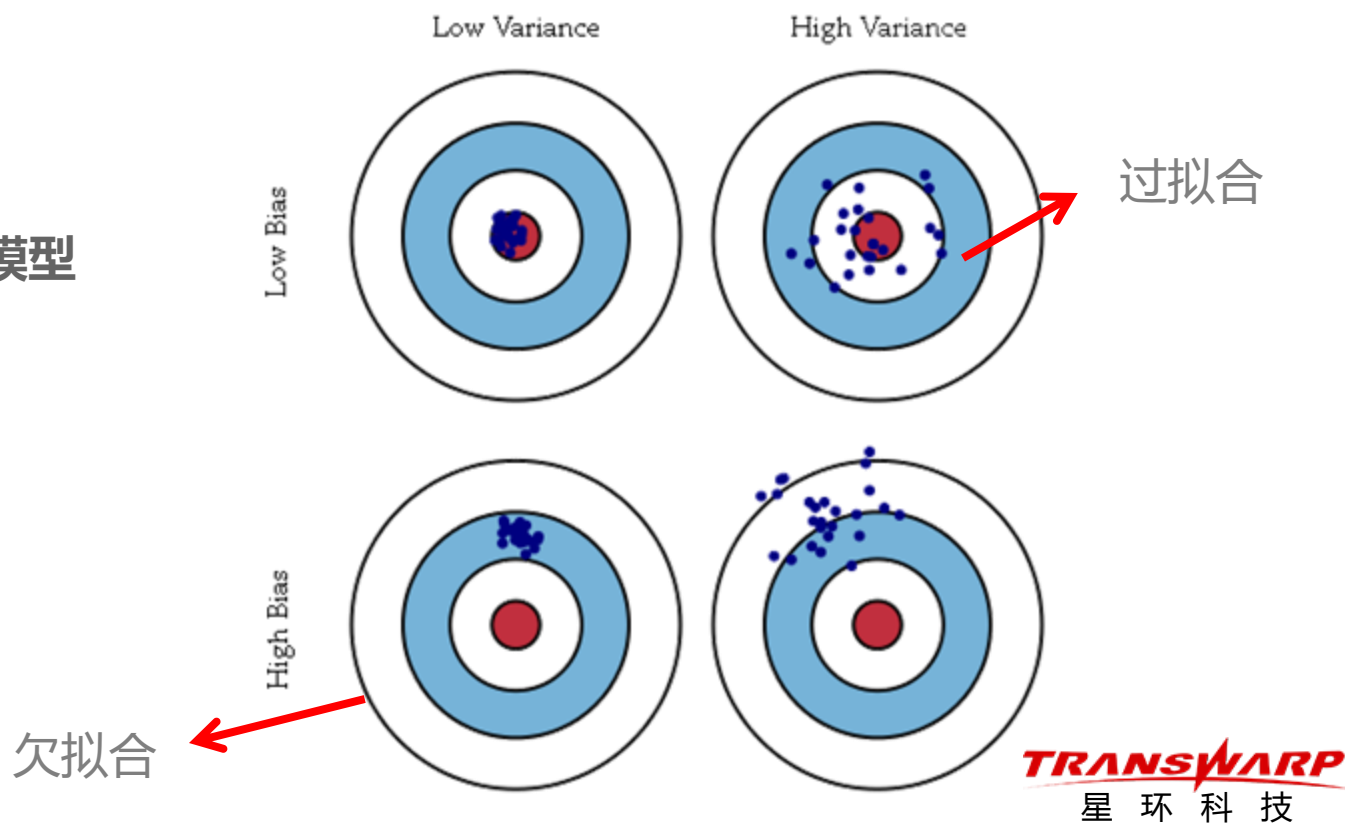
- **偏差**度量了学习算法的**期望预测**与**真实结果**的偏离程度，即刻画了**算法本身的拟合能力**。
- **方差**度量了同样大小的训练集的变动所导致的学习性能变化，即刻画了**数据扰动所造成的影响**。
- **噪声**则表达了在当前任务上任何学习算法所能达到的期望泛化误差的**下界**，即刻画了学习问题**本身的难度**。

前提条件：同一算法在不同训练集上训练的K个模型

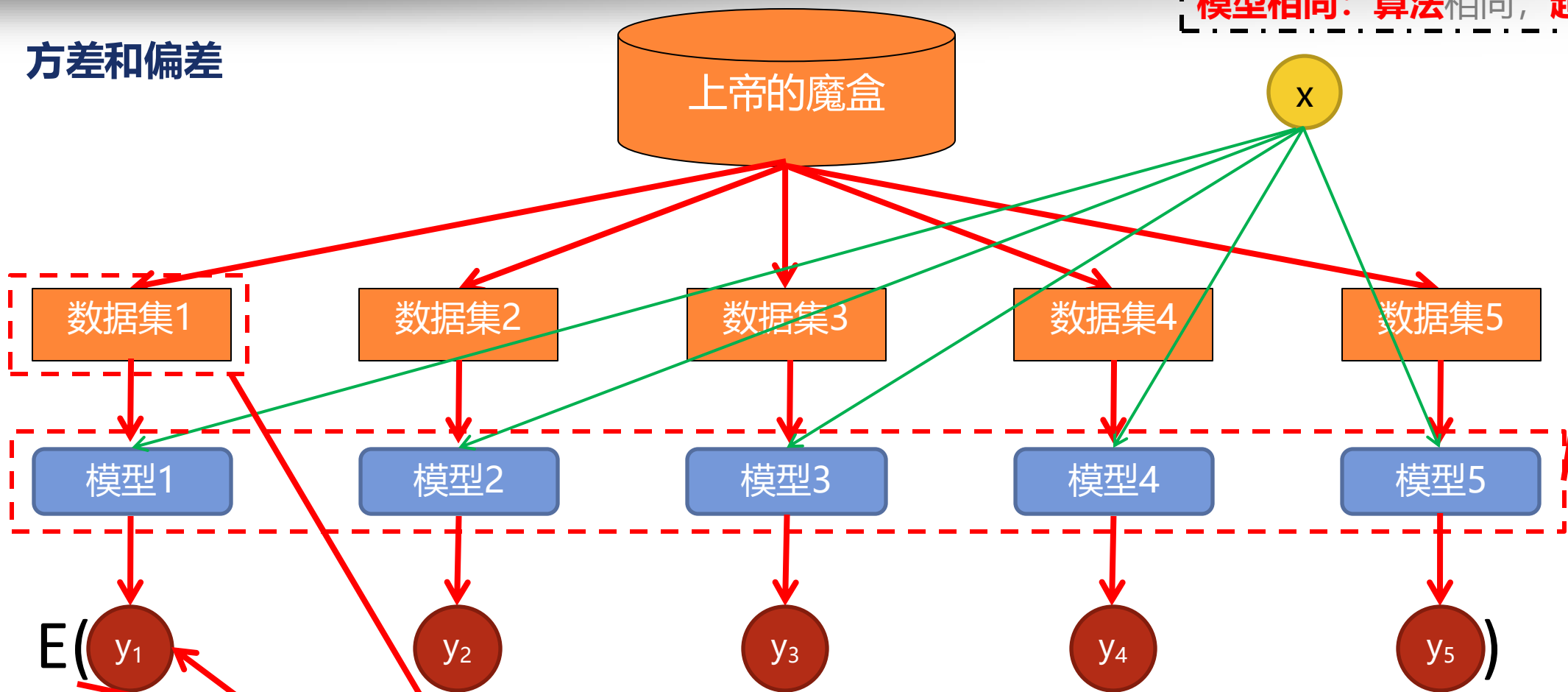
蓝点：K个模型预测同一个样本的预测值

靶心：该样本标注的真实值

泛化误差 $\approx \text{偏差}^2 + \text{方差} + \text{噪声}$



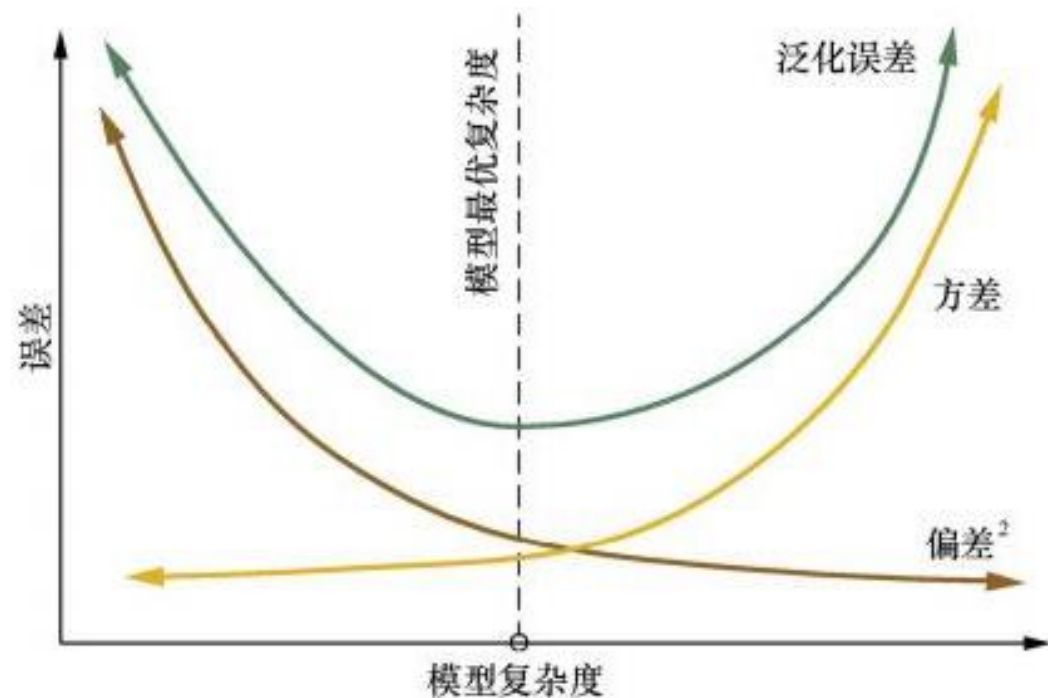
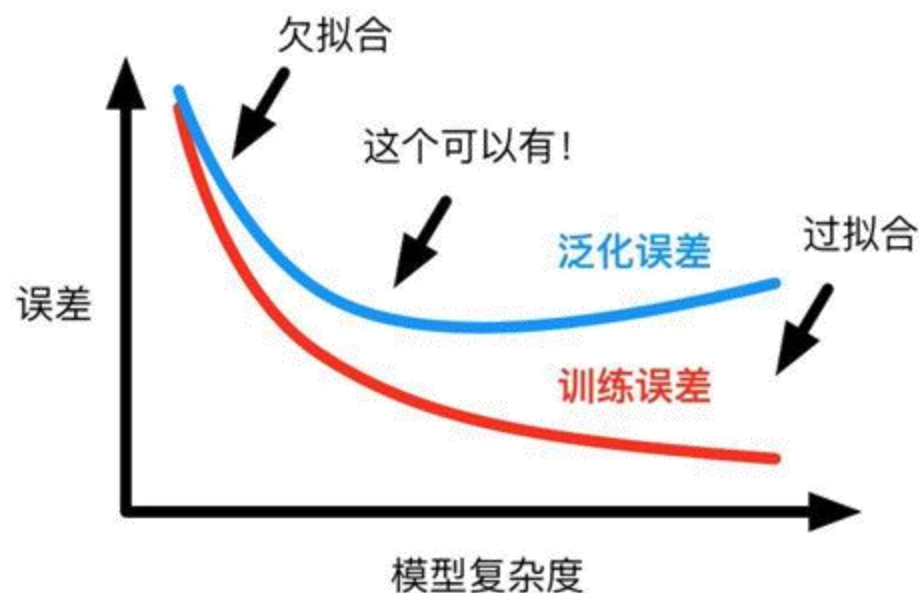
方差和偏差



$$\begin{aligned} &= \mathbb{E}_D [\{\bar{f}(x; \bar{D}) - \bar{f}(x)\}^2] + \{\bar{f}(x) - y\}^2 + \mathbb{E}_D [\{y - y_d\}^2] \\ &= \text{Variance} + \text{Bias} + \text{Noise} \end{aligned}$$

过拟合和欠拟合

- **过拟合**-高方差**低**偏差-模型复杂度**高**
- **欠拟合**-低方差**高**偏差-模型复杂度**低**



如何理解模型复杂度高？

- 算法方面：
 - 算法**参数**过多
 - 算法**拟合能力太强**
 - 正则**太弱**
 - 算法学习**过头**
- 数据方面
 - **数据太少**
 - **特征太多**

如何降低模型复杂度（解决过拟合）？

- 算法方面：
 - **减少**参数
 - 换**简单的**算法
 - **加强**正则
 - **早停**
- 数据方面
 - **数据增强**(变相增加数据)
 - **特征选择和特征降维**

小结

本节我们主要学习了过拟合和欠拟合以及方差和偏差：

- **过拟合**：训练误差**远低于**泛化误差，**高**方差**低**偏差，模型复杂度高
- **欠拟合**：训练误差和泛化误差**都很高**，**低**方差**高**偏差，模型复杂度低
- **目标**：追求**泛化误差最小化**





1 2 chapter

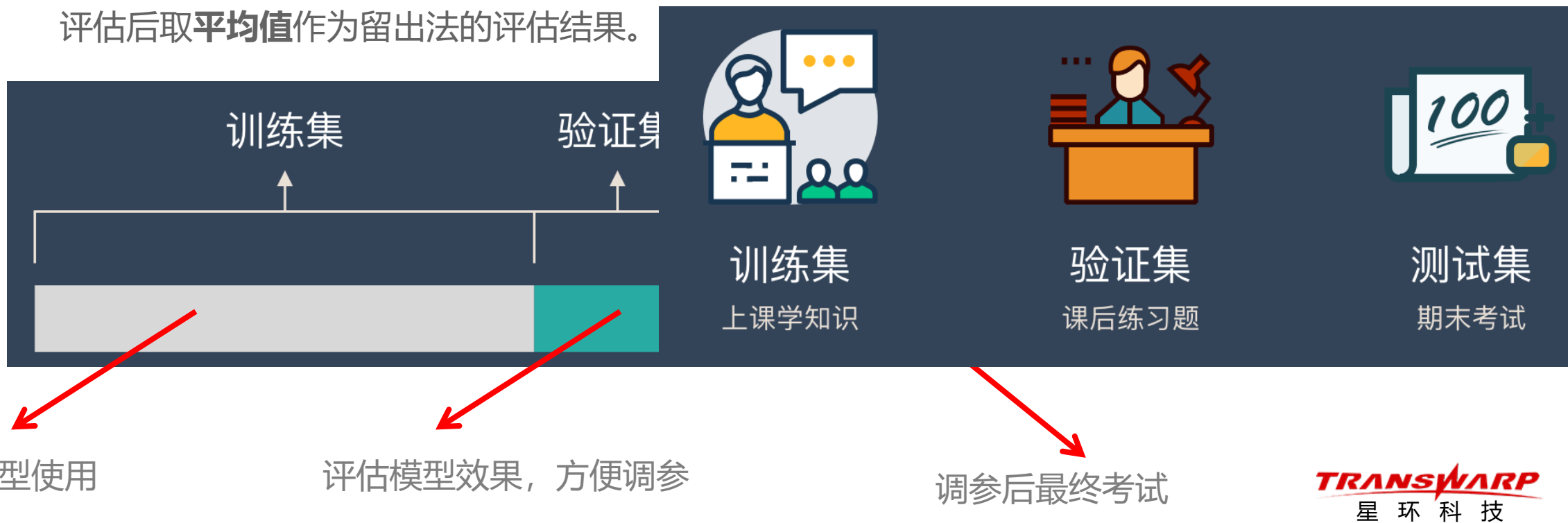
评估方法

测试集和测试误差

- 通常通过**实验测试**来对学习器的**泛化误差**进行评估。为此，需要使用一个“**测试集**” (testing set) 来测试学习器对新样本的判别能力，然后以测试集上的“**测试误差**” (testing error) 作为**泛化误差的近似**。
- **【重要假设】**：测试样本也是从样本真实分布中独立同分布采样而得。
- 常用评估方法：**留出法、交叉验证、自助法**

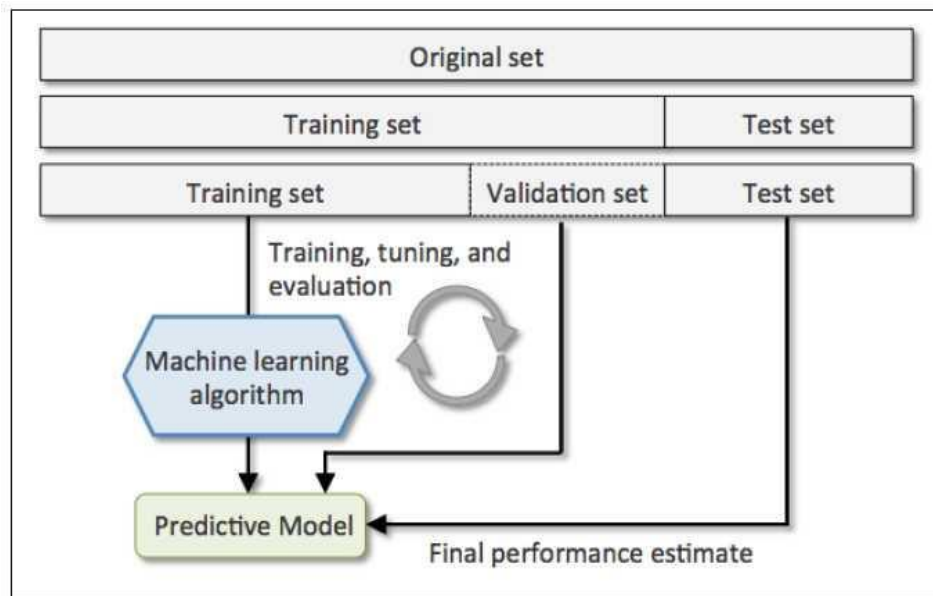
留出法

- 直接将整个数据集D划分为三个互斥的集合作为**训练集**、**验证集**和**测试集**。
- 注意：
 - 尽可能保持数据分布的一致性，一般**分层采样**（stratified sampling）
 - **单次**使用留出法得到的估计结果往往**不够稳定可靠**，一般要采用若干次随机划分、重复进行实验评估后取**平均值**作为留出法的评估结果。



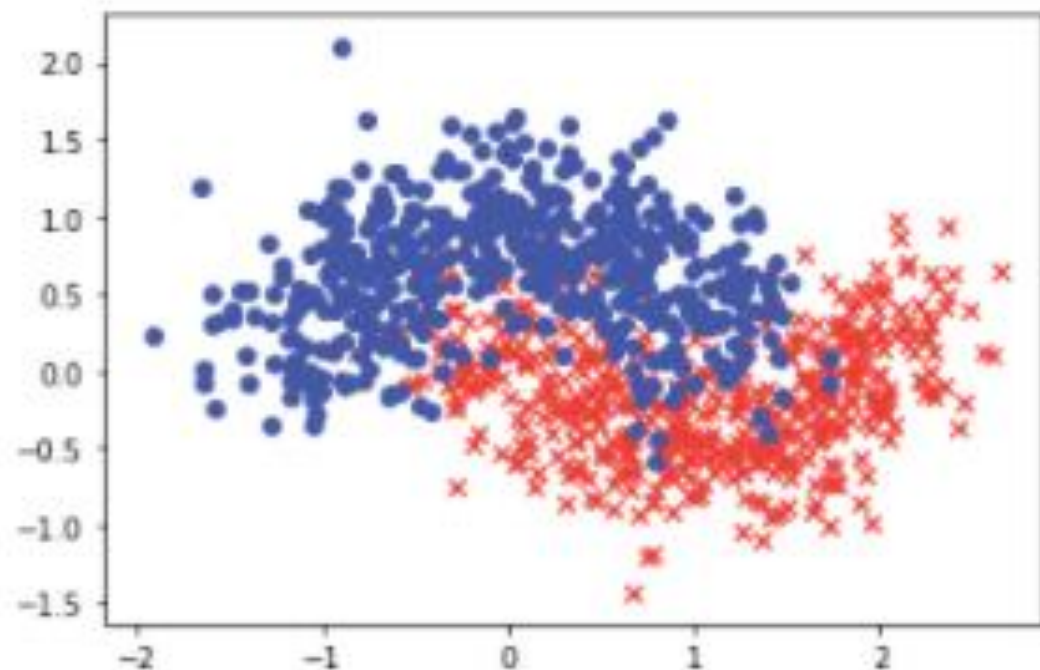
留出法 - 数据划分原则

- 数据划分的方法并没有明确的规定，不过可以参考3个原则：
 - 对于**小规模**样本集（几万量级），常用的分配比例是 60% 训练集、20% 验证集、20% 测试集。
 - 对于**大规模**样本集（百万级以上），只要验证集和测试集的数量足够即可，例如有 100w 条数据，那么留 1w 验证集，1w 测试集即可。1000w 的数据，同样留 1w 验证集和 1w 测试集。
 - 超参数**越少**，或者超参数很**容易**调整，那么可以减少验证集的比例，更多的分配给训练集。



导入工具包，构造数据

```
# 通过datasets.make_moons构造一个二分类数据集: 1000行, X有2个变量, y取值为0, 1: from sklearn.datasets import make_moons
X, y = make_moons( n_samples=1000, random_state=1, noise=0.3)
# 查看数据分布情况
import matplotlib.pyplot as plt
%matplotlib inline
plt.scatter(X[y == 1, 0], X[y == 1, 1], color="#BB3D34", marker="x")
plt.scatter(X[y == 0, 0], X[y == 0, 1], color="#4458A7", marker="o")
```



数据处理

```
# 对数据进行拆分, 分别分为训练集, 测试集, 验证集
from sklearn.model_selection import train_test_split
# 第一次拆分, 20%作为测试集
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=123456)
# 第二次拆分: 80%中的25%作为验证集
X_train, X_val, y_train, y_val = train_test_split(X_train, y_train, test_size=0.25, random_state=123456)
print(X_train.shape, X_val.shape, X_test.shape)
```

```
(600, 2) (200, 2) (200, 2)
```

模型进行训练及调参

```
# 使用K-近邻算法, 对K值进行调参, 使用验证集来评估误差, 筛选最佳K值。  
from sklearn.neighbors import KNeighborsClassifier  
# 对k值进行调参, 评估k值从1到20的模型正确率  
for k in range(20):  
    # 训练KNN模型  
    knn = KNeighborsClassifier(n_neighbors=k+1)  
    knn.fit(X_train, y_train)  
    # 用验证集评估模型的正确率  
    val_acc = knn.score(X_val, y_val)  
    print("k-%d, accuracy-%.3f"%(k+1, val_acc))
```

```
k-1, accuracy=0.845  
k-2, accuracy=0.875  
k-3, accuracy=0.865  
k-4, accuracy=0.875  
k-5, accuracy=0.885  
k-6, accuracy=0.895  
k-7, accuracy=0.890  
k-8, accuracy=0.895  
k-9, accuracy=0.895  
k-10, accuracy=0.900  
k-11, accuracy=0.900  
k-12, accuracy=0.905  
k-13, accuracy=0.905  
k-14, accuracy=0.905  
k-15, accuracy=0.915  
k-16, accuracy=0.915  
k-17, accuracy=0.915  
k-18, accuracy=0.915  
k-19, accuracy=0.915  
k-20, accuracy=0.910
```

模型评估

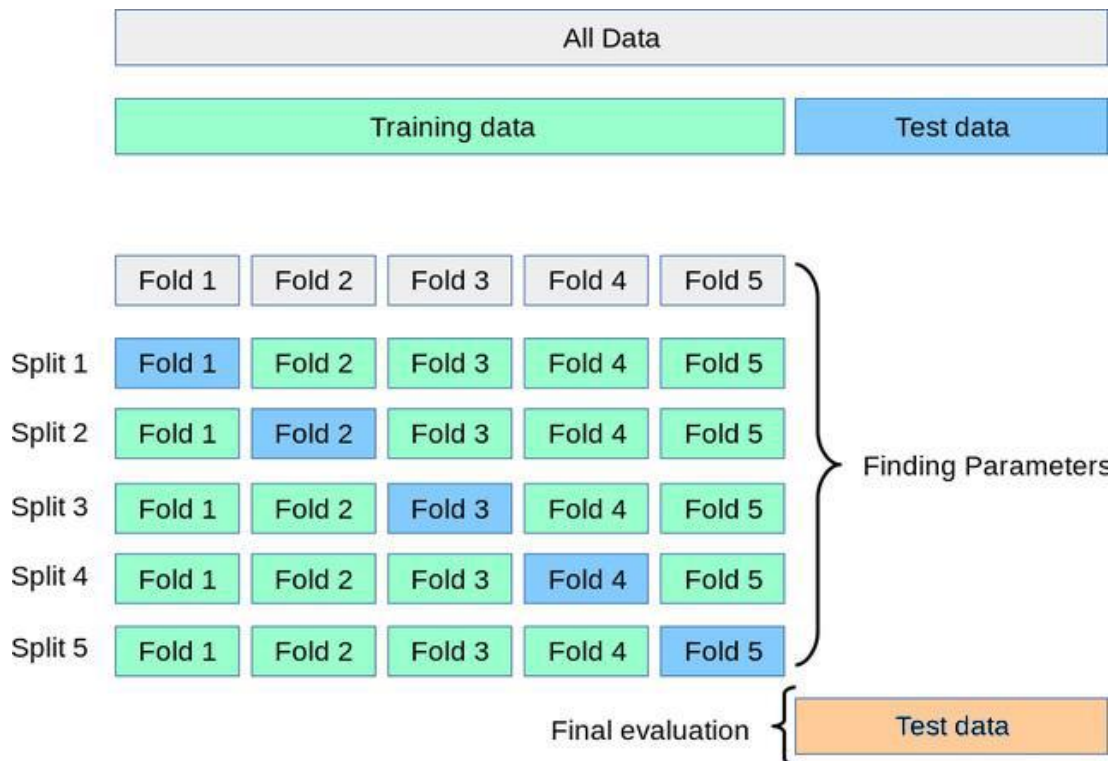
```
# 使用k=15构建最终的模型, 并用测试集评估  
knn = KNeighborsClassifier(n_neighbors=15)  
knn.fit(X_train, y_train)  
# 用测试集评估  
knn.score(X_test, y_test)
```

0.86

交叉验证

- 将数据集D划分为**k**个大小相似的**互斥子集**。
- 每次用**k-1**个子集的并集作为**训练集**，剩下的那一个子集用作**验证集**；最后返回**k**个**评估结果的均值**。
- 通常称为**k折交叉验证** (k-fold cross validation) 。k常取值为**10**。

更能够代表模型真实水平，**评估更可靠**



$$E = \frac{1}{10} \sum_{i=1}^{10} E_i$$

导入工具包，数据准备

```
from sklearn.datasets import make_moons  
X, y = make_moons( n_samples=1000, random_state=1, noise=0.3)
```

```
# 将数据拆分成训练集和测试集  
from sklearn.model_selection import train_test_split  
# 第一次拆分, 20%作为测试集  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=123456)
```

模型评估

```
# 用K折交叉验证评估模型
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_val_score
for k in range(20):
    # 用10折交叉验证评估不同KNN算法不同k值的模型正确率
    knn = KNeighborsClassifier(n_neighbors=k+1)
    cv_acc = cross_val_score(knn, X_train, y_train, cv=10).mean()
    print("k-%d, accuracy-%.3f"%(k+1, cv_acc))
```

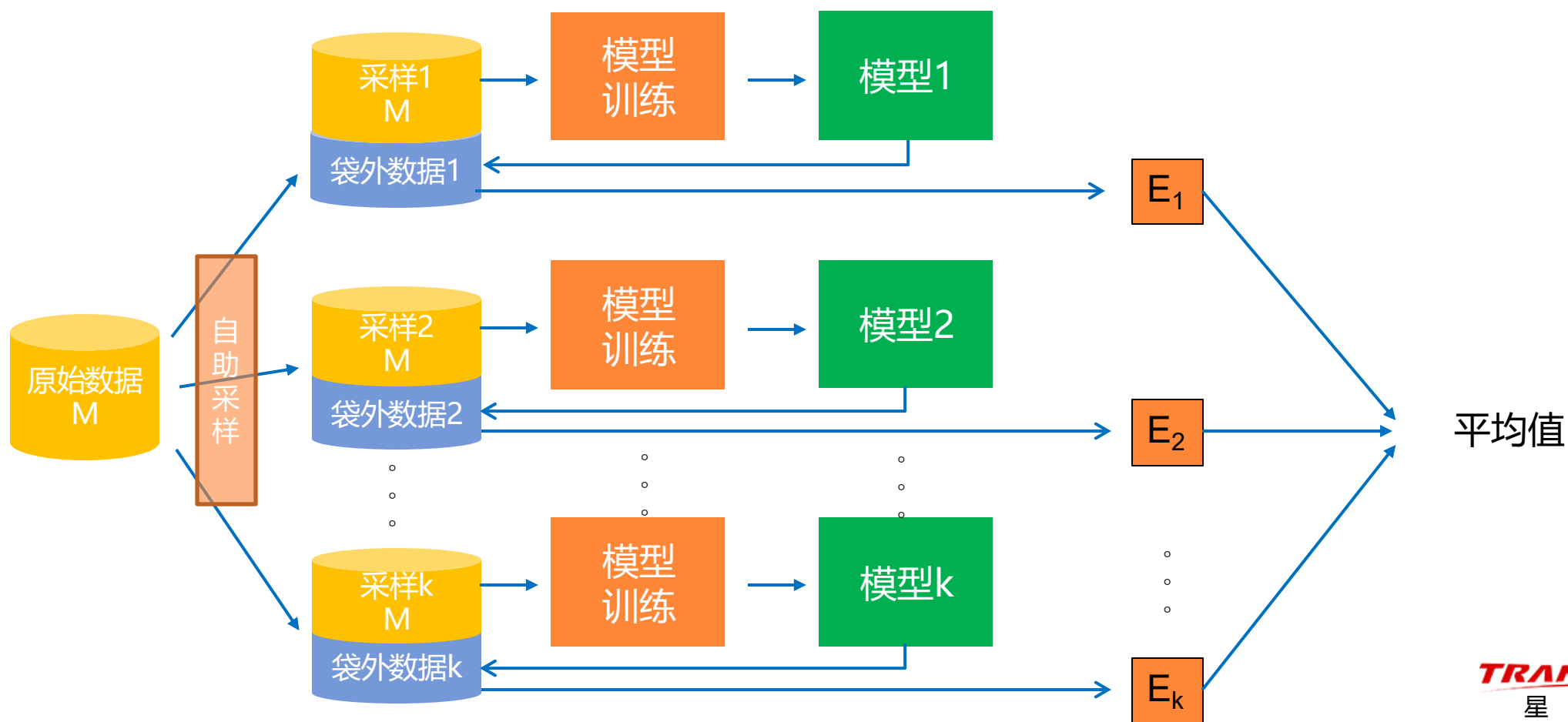
```
# 测试集评估模型
# Step4: 用测试集评估模型
# 使用k=14构建最终的模型，并用测试集评估
knn = KNeighborsClassifier(n_neighbors=14)
knn.fit(X_train, y_train)
# 用测试集评估
knn.score(X_test, y_test)
```

```
k-1, accuracy-0.870
k-2, accuracy-0.877
k-3, accuracy-0.882
k-4, accuracy-0.875
k-5, accuracy-0.892
k-6, accuracy-0.891
k-7, accuracy-0.888
k-8, accuracy-0.898
k-9, accuracy-0.903
k-10, accuracy-0.902
k-11, accuracy-0.903
k-12, accuracy-0.904
k-13, accuracy-0.905
k-14, accuracy-0.906
k-15, accuracy-0.899
k-16, accuracy-0.904
k-17, accuracy-0.901
k-18, accuracy-0.902
k-19, accuracy-0.900
k-20, accuracy-0.905
```

0.855

自助法 (bootstrapping)

- 自助法以**自助采样**为基础（有放回采样）。每次随机从D中挑选一个样本，放入D'中，然后将样本放回D中，重复m次之后，得到了包含m个样本的数据集。



小结

本节我们主要学习了**留出法**、**k折交叉验证**和自助法等评估方法：

- **留出法**和**k折交叉验证**更常用
- 训练集：用于模型训练
- 验证集：用于评估模型并进行调参（多次评估）
- 测试集：最终评估（只能一次）



1
3
chapter

性能度量

性能度量概述

- 所有事情都需要评估好坏，尤其是量化的评估指标。
 - **高考成绩**用来评估学生的学习能力
 - **杠铃的重量**用来评估肌肉的力量
 - **跑分**用来评估手机的综合性能
- 机器学习有很多**评估的指标**。有了这些指标我们就**横向的比较**哪些模型的**表现更好**。

性能度量概述

机器学习评估指标大全

分类问题（单项）

准确率 – Accuracy

精确率 – Precision

召回率 – Recall

分类问题（综合）

F1分数

ROC曲线

AUC

回归问题

MAE

MSE

R2值

分类问题指标-混淆矩阵

- **一个栗子**：我们有10张照片，5张男性、5张女性



分类问题指标：混淆矩阵

- 有一个判断性别的机器学习模型，当我们使用它来判断「**是否为男性**」时（男为**正例**，女为**负例**），会出现4种情况：



让机器判断是否为男性

- 男判男, True
- 男判女, False
- 女判女, True
- 女判男, False

分类问题指标：混淆矩阵

■ 这4种情况构成了经典的混淆矩阵：

- ✓ **TP** – True Positive: 男判男, True
- ✓ **FN** – False Negative: 男判女, False
- ✓ **TN** – True Negative: 女判女, True
- ✓ **FP** – False Positive: 女判男, False



	实际：男	实际：女
预测：男	 机器：男 TP 3	 机器：男 FP 1
预测：女	 机器：女 FN 2	 机器：女 TN 4

分类问题指标：准确率 – Accuracy

- 预测**正确的结果**占总样本的**百分比**:

	实际：男	实际：女
预测：男	 TP 3 机器：男	 FP 1 机器：男
预测：女	 FN 2 机器：女	 TN 4 机器：女

$$\text{准确率} = (TP + TN) / (TP + TN + FP + FN)$$

准确率
Accuracy

=


机器：男

+


机器：女

+


机器：男

+


机器：女

+


机器：女

+


机器：男

- **Bug**: 100个选择题 (99个A, 一个B), 小明全蒙A, 准确率99%
- **样本不平衡**导致准确率失效

分类问题指标：精确率(查准率) – Precision

- 所有被**预测为正的样本**中**实际为正**的样本的**概率**

$$\text{精准率} = \text{TP} / (\text{TP} + \text{FP})$$

精确率
Precision

=



	实际：男	实际：女
预测：男	 机器：男	 机器：男
预测：女	 机器：女	 机器：女

精确率和准确率看上去有些类似，但是**完全不同的**两个概念。精确率代表对**正样本结果中的预测准确程度**，而准确率则代表**整体的预测准确程度**，既包括正样本，也包括负样本

分类问题指标：召回率(查全率) – Recall

- **实际为正**的样本中被**预测为正**样本的**概率**

$$\text{召回率} = \text{TP} / (\text{TP} + \text{FN})$$

找回率
Recall

=

 机器：男

+

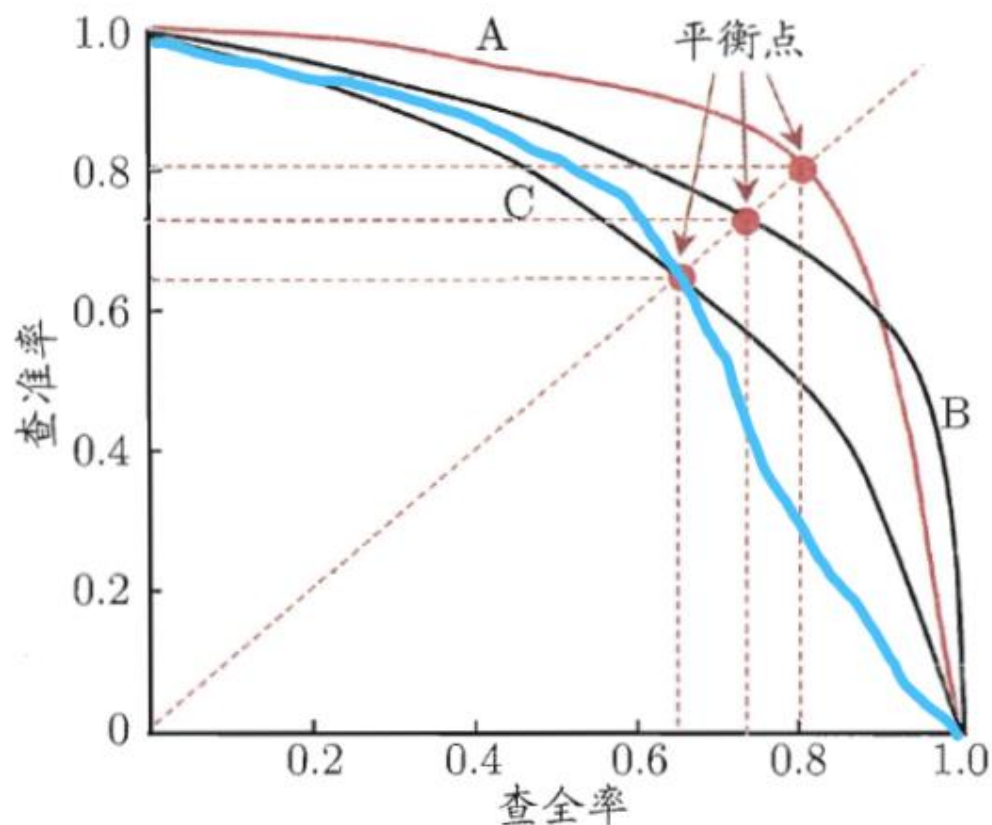
 机器：男  机器：女

	实际：男	实际：女
预测：男	 机器：男 TP 3	 机器：男 FP 1
预测：女	 机器：女 FN 2	 机器：女 TN 4

召回率越高，代表**实际正例**（坏用户）被预测出来的**概率越高**，它的含义类似：**宁可错杀一千，绝不放过一个。**

分类问题指标：F1 – score

- 如果我们把**精确率 (Precision)** 和**召回率 (Recall)** 之间的关系用图来表达，就是下面的**PR曲线**：

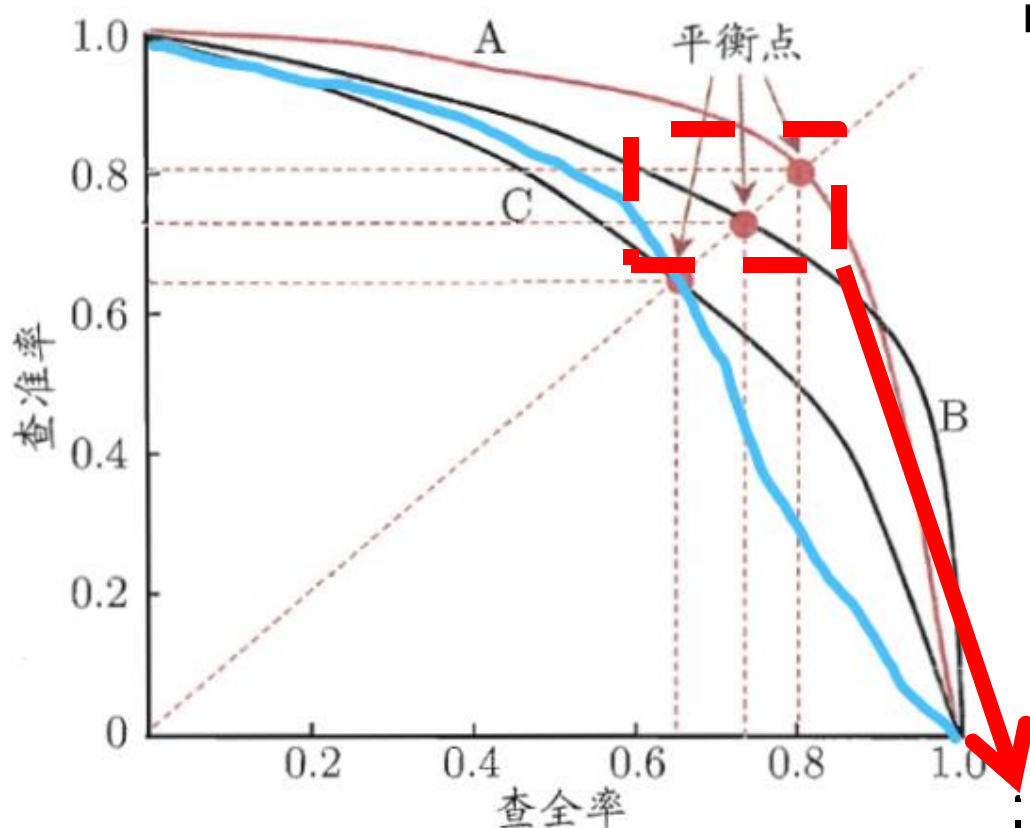


概率降序排序

y_p	y	
0.999	1	← (P _n , R _n)
0.998	0	
0.9	1	
0.8	1	
0.7	1	
0.68	1	
0.58	0	
0.52	0	
0.48	1	
0.45	0	
0.39	0	← (P ₄ , R ₄)
0.35	1	
0.25	0	← (P ₃ , R ₃)
0.19	0	
0.1	0	← (P ₂ , R ₂)
0.08	1	
0.03	0	← (P ₁ , R ₁)

分类问题指标：F1 – score

- 如果我们把**精确率 (Precision)** 和**召回率 (Recall)** 之间的关系用图来表达，就是下面的**PR曲线**：



- 如果可以发现他们俩的关系是「**两难全**」的关系。为了**综合两者**的表现，在两者之间找一个平衡点，就出现了一个 **F1 分数**。

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

$$F1 \text{ 分数} = \frac{2 \times \text{精确率} \times \text{召回率}}{\text{精确率} + \text{召回率}}$$

二者都高F1才高

分类问题指标：ROC/AUC 的概念

■ 灵敏度 (Sensitivity) = $TP / (TP + FN)$

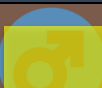
■ 特异度 (Specificity) = $TN / (FP + TN)$

	实际：男	实际：女
预测：男	 机器：男 TP	 机器：男 FP
预测：女	 机器：女 FN	 机器：女 TN

■ 真正率 (TPR) = 灵敏度 = $TP / (TP + FN)$

■ 假正率 (FPR) = $1 - \text{特异度} = FP / (FP + TN)$

我们发现 TPR 和 FPR 分别是基于实际表现 1 和 0 出发的，也就是说它们分别在实际的**正样本和负样本中**来观察相关概率问题。**无视样本不平衡**

	实际：男	实际：女
预测：男	 机器：男 TP	 机器：男 FP
预测：女	 机器：女 FN	 机器：女 TN

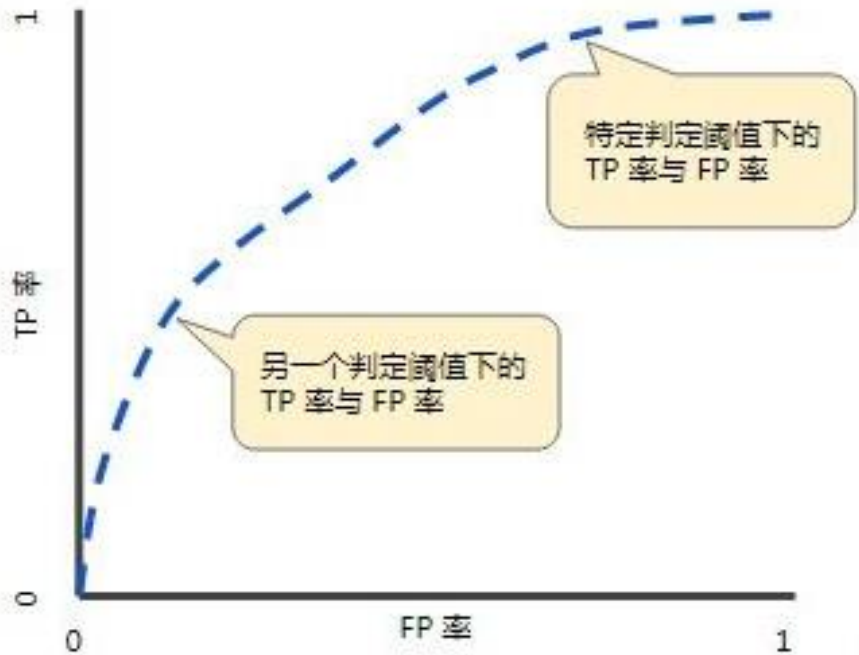
TPR

	实际：男	实际：女
预测：男	 机器：男 TP	 机器：男 FP
预测：女	 机器：女 FN	 机器：女 TN

FPR

分类问题指标：ROC（接受者操作特征曲线）

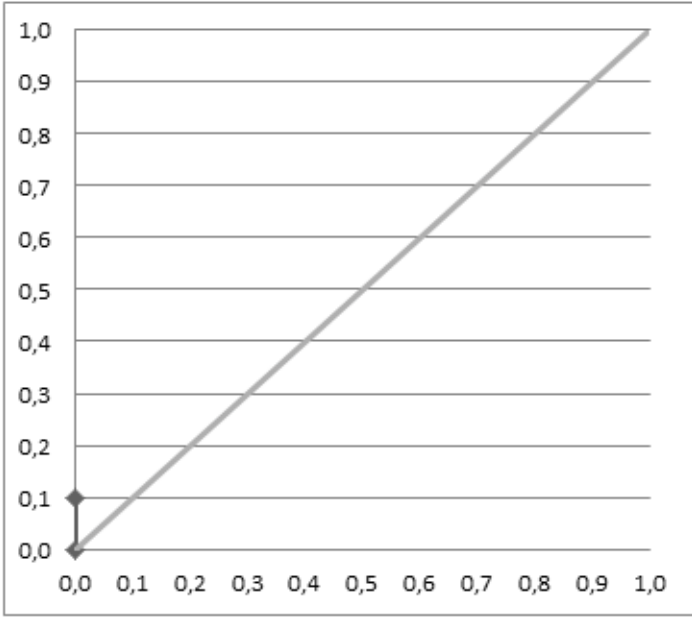
- ROC 曲线中的主要两个指标就是真正率和假正率，上面也解释了这么选择的好处所在。其中横坐标为假正率（FPR），纵坐标为真正率（TPR），下面就是一个标准的 ROC 曲线图。



概率降序排序

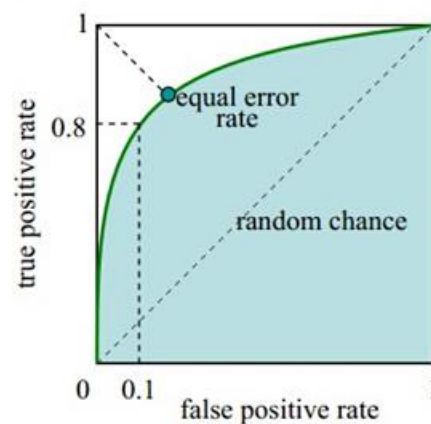
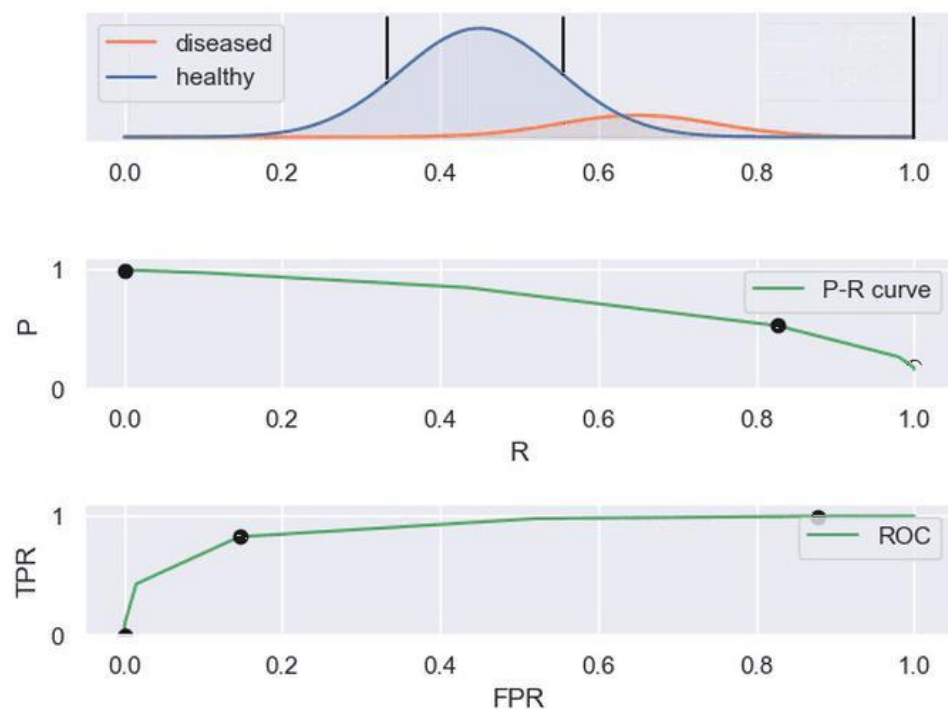
y_p	y	
0.999	1	← (FPR _n , TPR _n)
0.998	0	
0.9	1	
0.8	1	
0.7	1	
0.68	1	
0.58	0	
0.52	0	
0.48	1	
0.45	0	
0.39	0	← (FPR ₄ , TPR ₄)
0.35	1	
0.25	0	← (FPR ₃ , TPR ₃)
0.19	0	
0.1	0	← (FPR ₂ , TPR ₂)
0.08	1	
0.03	0	← (FPR ₁ , TPR ₁)

#	C	Score
1	P	0.9
2	P	0.8
3	N	0.7
4	P	0.6
5	P	0.55
6	P	0.54
7	N	0.53
8	N	0.52
9	P	0.51
10	N	0.505
11	P	0.4
12	N	0.39
13	P	0.38
14	N	0.37
15	N	0.36
16	N	0.35
17	P	0.34
18	N	0.33
19	P	0.3
20	N	0.1

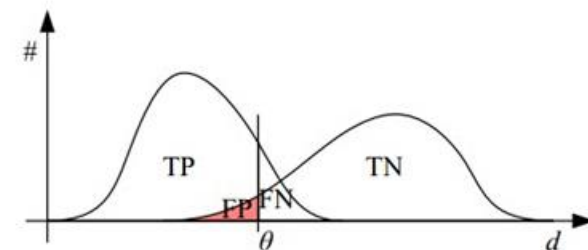


分类问题指标：ROC 曲线的阈值问题

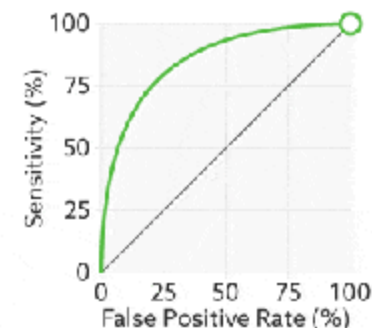
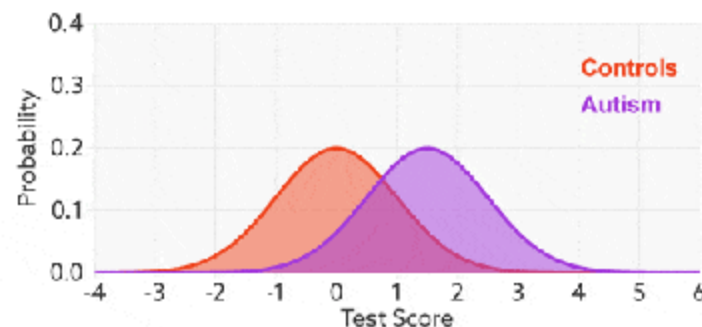
- 与前面的P-R 曲线类似，ROC 曲线也是通过**遍历所有阈值**来绘制整条曲线的。如果我们不断的遍历所有阈值，预测的正样本和负样本是在不断变化的，相应的在ROC 曲线图中也会沿着曲线滑动。



(a)

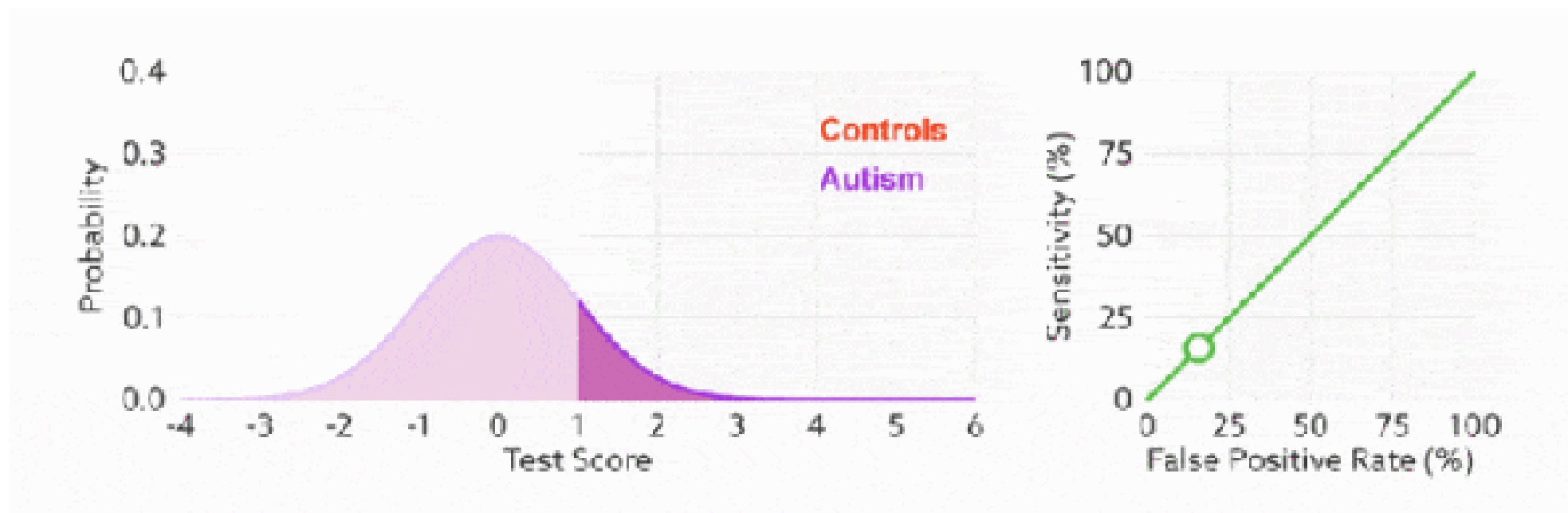


(b)



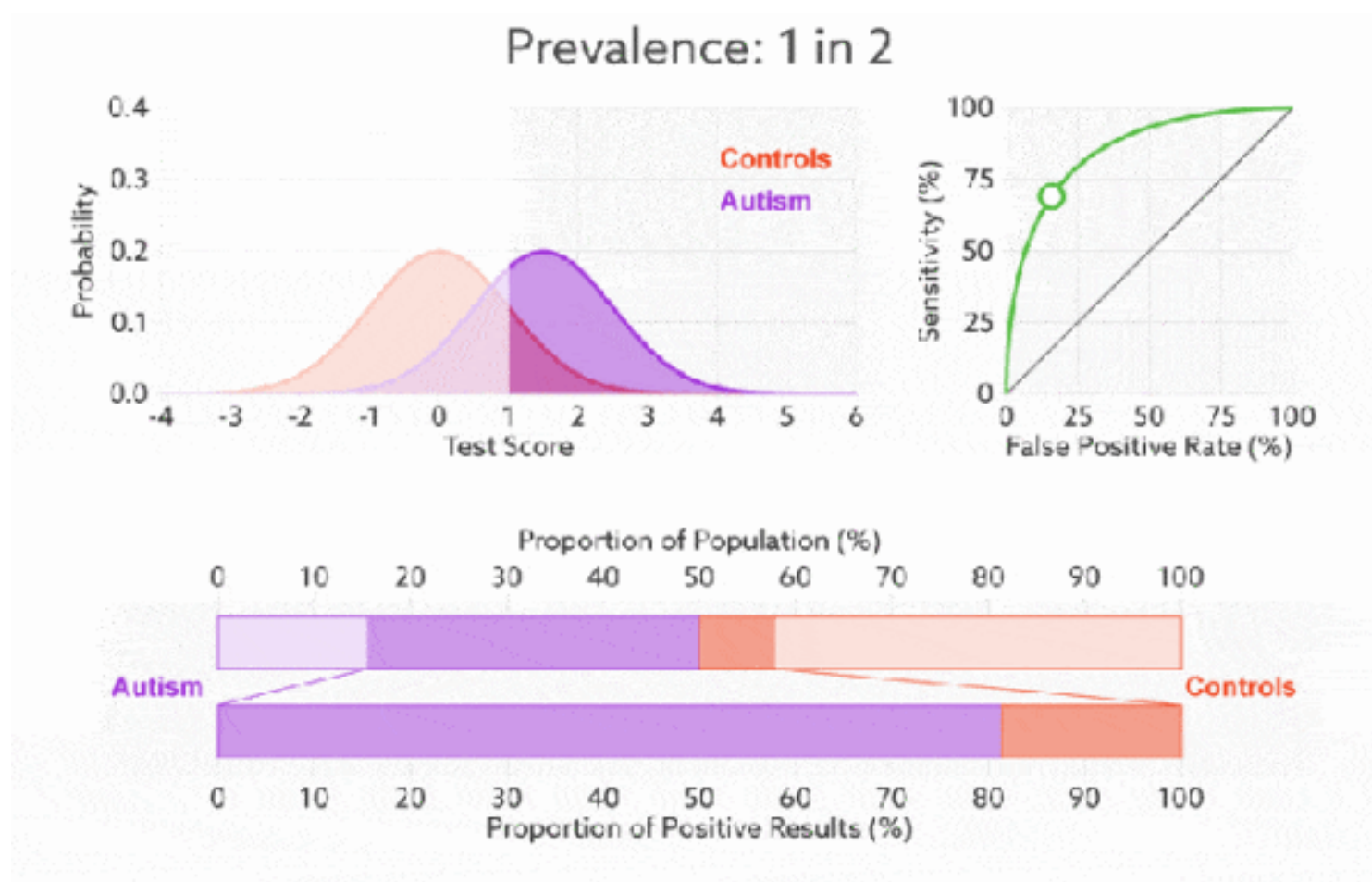
分类问题指标：如何判断 ROC 曲线的好坏？

- 改变阈值只是不断地改变预测的正负样本数，即 TPR 和 FPR，但是**曲线本身是不会变的**。那么如何判断一个模型的 ROC 曲线是好的呢？
- TPR 越高，同时 FPR 越低（即 ROC 曲线越陡），那么模型的性能就越好



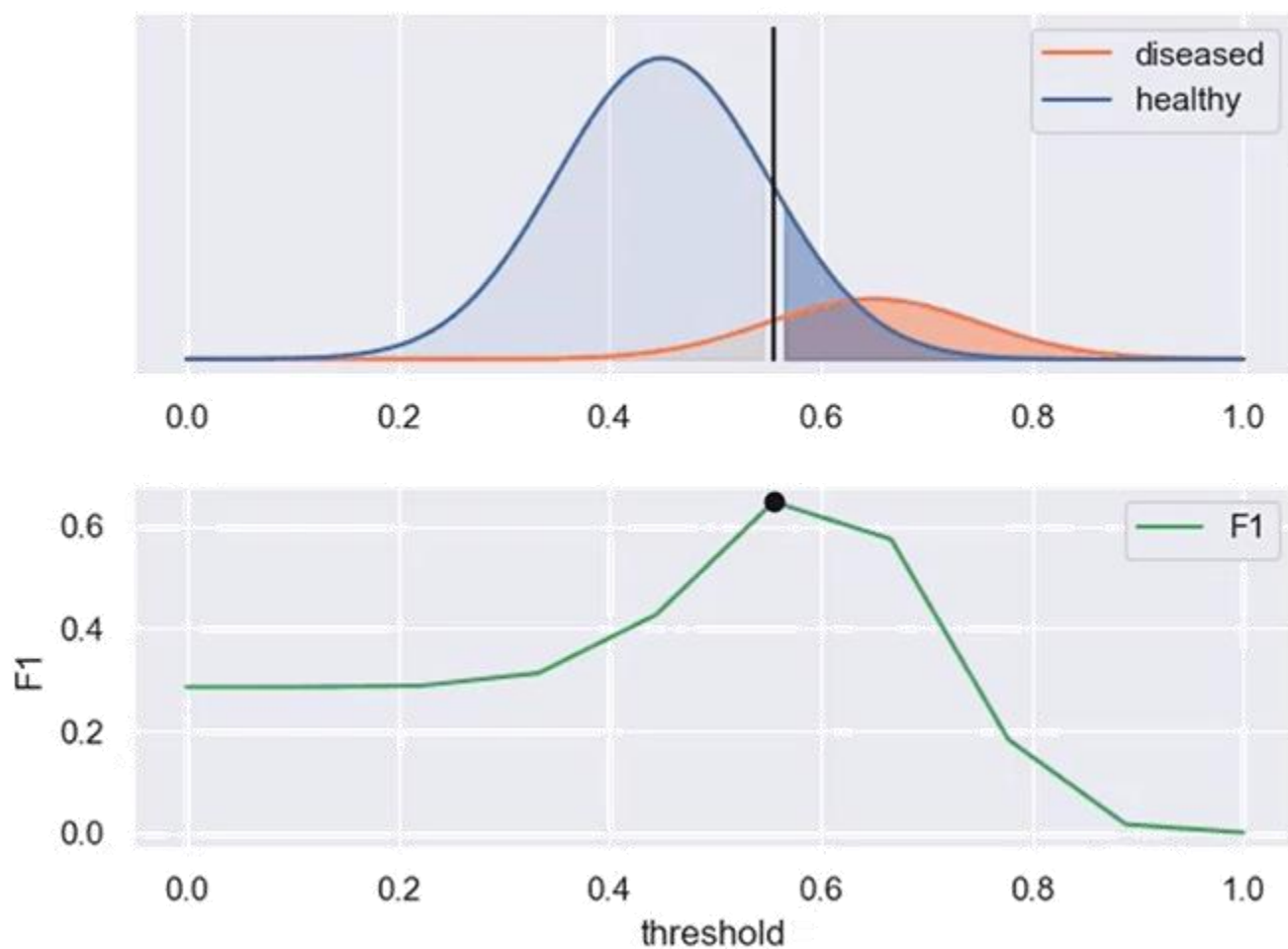
分类问题指标：ROC 曲线无视样本不平衡

- 前面已经对 ROC 曲线为什么可以无视样本不平衡做了解释，下面我们用动态图的形式再次展示一下它是如何工作的。我们发现：无论红蓝色样本比例如何改变，ROC 曲线都没有影响。



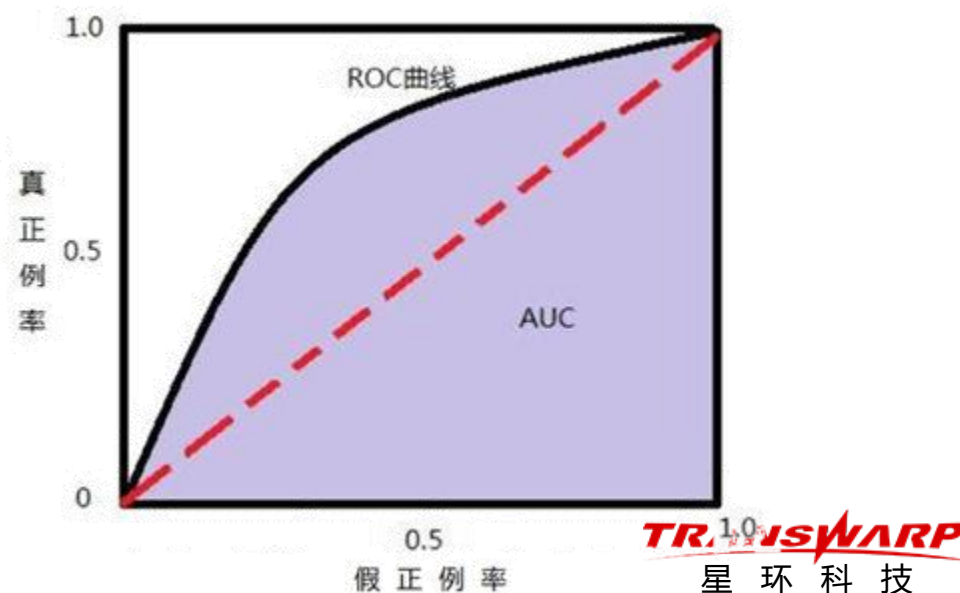
分类问题指标：怎么确定阈值？

- 取**F1最高值**所对应的阈值



分类问题指标：AUC（曲线下的面积）

- ROC 曲线计算效率非常低。有一种**基于排序的高效算法**可以替代 ROC，这种算法称为**曲线下面积 (Area Under Curve)**
- AUC 意义：
 - ROC 曲线下面积 (Area Under Curve)
 - **正样本概率大于负样本概率的概率**
- **对角线**：表示**随机瞎猜效果**。
- 判断标准：
 - 0.5 - 0.7：效果较低，但用于预测股票已经很不错了
 - 0.7 - 0.85：效果一般
 - 0.85 - 0.95：效果很好
 - 0.95 - 1：效果非常好，但一般不太可能



回归问题指标：AUC（曲线下的面积）

$$MAE = \frac{1}{m} \sum_{i=1}^m |y_i - \hat{y}_i|$$

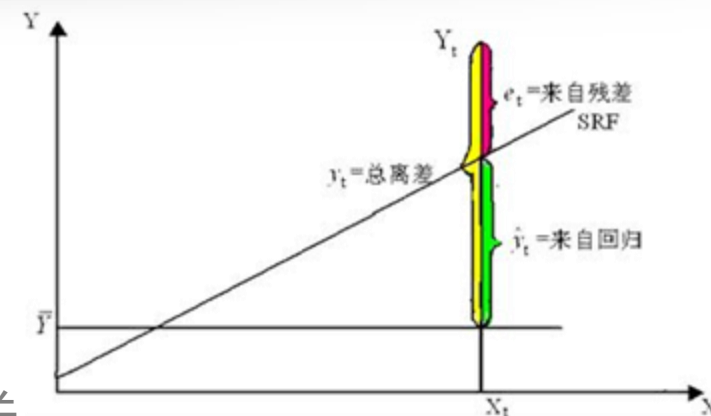
$$MSE = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$$

$$RMSE = \sqrt{MSE} = \sqrt{\frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2}$$

$$R^2 = 1 - \frac{RSS}{TSS} = 1 - \frac{\sum_{i=1}^m (y_i - \hat{y}_i)^2}{\sum_{i=1}^m (y_i - \bar{y})^2}$$

$$\bar{y} = \frac{1}{m} \sum_{i=1}^m y_i$$

越小越好，跟y的方差有关



$$y = \theta^T x + \varepsilon$$

真实值

预测值

误差

总方差

可解释方差

不可解释方差

可解释方差比例，0-1之间，越大越好

不可解释方差比例

导入工具包，加载数据

```
# 导入相关包
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report
from sklearn.metrics import roc_curve
from sklearn.metrics import auc

# 读取数据
titanic_df = pd.read_csv('train.csv')

# drop掉PassengerId, Name, Ticket, Cabin
titanic_df = titanic_df.drop(['PassengerId', 'Name', 'Ticket', 'Cabin'], axis = 1)

# 构造一个新变量AgeIsMissing
titanic_df['AgeIsMissing'] = 0
titanic_df.loc[titanic_df['Age'].isnull(), 'AgeIsMissing'] = 1
```

数据处理

```
# 对Age缺失值进行均值填充
age_mean = round(titanic_df['Age'].mean())
titanic_df['Age'].fillna(age_mean, inplace=True)

# 对Embarked缺失值用'S'替换
titanic_df['Embarked'].fillna('S', inplace=True)

# 对Age进行分箱—自定义分箱
cut_points = [0, 18, 25, 40, 60, 100]
titanic_df["AgeBin"] = pd.cut(titanic_df.Age, bins=cut_points)

# 对Fare船票价格进行分箱—等深分箱
titanic_df["FareBin"] = pd.qcut(titanic_df.Fare, 5)

# 构造FamilySize变量
titanic_df['FamilySize'] = titanic_df['SibSp'] + titanic_df['Parch'] + 1

# 构造一个新变量IsAlone (是否独自一人)
titanic_df['IsAlone'] = 0
titanic_df.loc[titanic_df['FamilySize'] == 1, 'IsAlone'] = 1

# 构造一个新变量IsMother (是否是母亲)
titanic_df['IsMother'] = 0
titanic_df.loc[(titanic_df['Sex'] == 'female') & (titanic_df['Parch'] > 0) & (titanic_df['Age'] > 20), 'IsMother'] = 1

# 把Sex性别和AgeBin特征进行组合
titanic_df['SexAgeCombo'] = titanic_df['Sex'] + "_" + titanic_df['AgeBin'].astype(str)
```

数据处理

```
# 对Pclass, Sex, Embarked, AgeBin, FareBin, FamilySize, Sex_Age_combo进行独热编码
```

```
Pclass = pd.get_dummies(titanic_df.Pclass, prefix='Pclass')
```

```
Sex = pd.get_dummies(titanic_df.Sex, prefix='Sex')
```

```
Embarked = pd.get_dummies(titanic_df.Embarked, prefix='Embarked')
```

```
AgeBin = pd.get_dummies(titanic_df.AgeBin, prefix='AgeBin')
```

```
FareBin = pd.get_dummies(titanic_df.FareBin, prefix='FareBin')
```

```
FamilySize = pd.get_dummies(titanic_df.FamilySize, prefix='FamilySize')
```

```
SexAgeCombo = pd.get_dummies(titanic_df.SexAgeCombo, prefix='SexAgeCombo')
```

```
# 把需要的变量全部拼接在一起
```

```
TrainData = pd.concat([titanic_df[['Survived', 'AgeIsMissing', 'IsAlone', 'IsMother']],  
                       Pclass, Sex, Embarked, AgeBin, FareBin, FamilySize, SexAgeCombo], axis=1)
```

```
# 将数据集拆分成训练集和测试集
```

```
trainData_X = TrainData.drop(['Survived'], axis = 1)
```

```
trainData_y = TrainData.Survived
```

```
X_train, X_test, y_train, y_test = train_test_split(trainData_X, trainData_y, test_size=0.3, random_state=123456)
```

模型训练

```
# 构建逻辑回归模型
```

```
# 实例化一个逻辑回归模型lr
```

```
lr = LogisticRegression()
```

```
# 训练模型
```

```
lr.fit(X_train, y_train)
```

```
LogisticRegression(C=1.0, class_weight=None, dual=False, fit_intercept=True,  
                    intercept_scaling=1, l1_ratio=None, max_iter=100,  
                    multi_class='warn', n_jobs=None, penalty='l2',  
                    random_state=None, solver='warn', tol=0.0001, verbose=0,  
                    warm_start=False)
```

模型评估

```
# 创建误分类矩阵, 进行模型评估

# 对测试集X_test进行预测, 得到y_pred
y_pred = lr.predict(X_test)
# 误分类矩阵
confusion_matrix(y_test, y_pred)

array([[134,  32],
       [ 29,  73]], dtype=int64)

# 优化代码, 将误分类矩阵可视化, 更加直观

# 定义了一个更美观误分类矩阵的函数
def show_confusion_matrix(cnf_matrix, class_labels):
    plt.matshow(cnf_matrix, cmap=plt.cm.YlGn, alpha=0.7)
    ax = plt.gca()
    ax.set_xlabel('Predicted Label', fontsize=16)
    ax.set_xticks(range(0, len(class_labels)))
    ax.set_xticklabels(class_labels, rotation=45)
    ax.set_ylabel('Actual Label', fontsize=16, rotation=90)
    ax.set_yticks(range(0, len(class_labels)))
    ax.set_yticklabels(class_labels)
    ax.xaxis.set_label_position('top')
    ax.xaxis.tick_top()

    for row in range(len(cnf_matrix)):
        for col in range(len(cnf_matrix[row])):
            ax.text(col, row, cnf_matrix[row][col], va='center', ha='center', fontsize=16)
```

模型评估

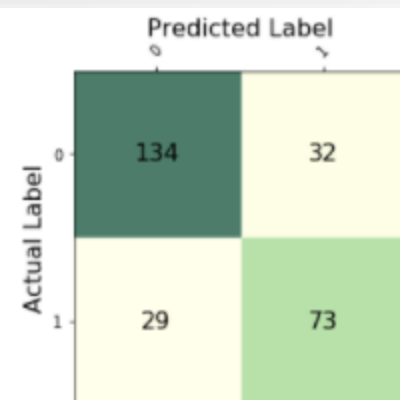
```
# 美观显示混淆矩阵
```

```
class_labels = [0, 1]
```

```
cnf_matrix = confusion_matrix(y_test, y_pred)
```

```
show_confusion_matrix(cnf_matrix, class_labels)
```

```
plt.show()
```



```
# 计算正确率
```

```
# 对测试集X_test进行预测, 得到y_pred
```

```
y_pred = lr.predict(X_test)
```

```
# 正确率
```

```
accuracy_score(y_test, y_pred)
```

0.7723880597014925

```
# 计算准确率, 召回率, F1分数
```

```
# 对测试集X_test进行预测, 得到
```

```
y_predy_pred = lr.predict(X_test)
```

```
# 分类评估报告classification_report
```

```
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.82	0.81	0.81	166
1	0.70	0.72	0.71	102
accuracy			0.77	268
macro avg	0.76	0.76	0.76	268
weighted avg	0.77	0.77	0.77	268

模型评估

```
# 计算ROC曲线和AUC
```

```
# 对测试集X_test进行预测, 得到正例的预测概率
```

```
y_prob = lr.predict_proba(X_test)[:,1] #由于输出概率有2个, 我们只取第2个 (标签=1的概率)
```

```
# 计算ROC曲线
```

```
fpr, tpr, thresholds = roc_curve(y_test, y_prob)
```

```
# 计算AUC
```

```
auc = auc(fpr, tpr)
```

```
auc
```

```
0.8128395936687928
```

```
# 绘制ROC曲线
```

```
plt.plot(fpr, tpr, lw=2, label='ROC curve (area = {:.2f})'.format(auc))
```

```
plt.plot([0, 1], [0, 1], 'r--')
```

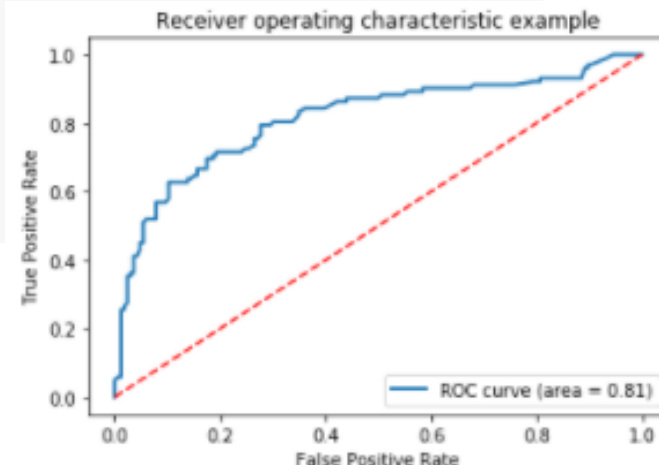
```
plt.xlabel('False Positive Rate')
```

```
plt.ylabel('True Positive Rate')
```

```
plt.title('Receiver operating characteristic example')
```

```
plt.legend(loc='lower right')
```

```
plt.show()
```



小结

本节我们主要学习了**分类模型评估**和回归模型评估方法：

■ 分类模型评估指标

- 混淆矩阵：TP, FP, TN, FN
- 准确率 = $(TP + TN) / (TP + TN + FP + FN)$
- 精确率 = $TP / (TP + FP)$
- 召回率 = $TP / (TP + FN)$
- $F1 = 2 * P * R / (P + R)$ (受样本不平衡影响)
- ROC曲线和AUC (不受样本不平衡影响)

■ 回归模型评估：MAE、MSE, RMSE, R2



以下两种情境，分别对应分类评价中的哪两个指标？

- (a) 被预测为小偷的人中，实际是小偷的比例；
- (b) 实际是小偷的人中，被成功识别为小偷的比例。

A. Precision, Recall

B. Recall, Precision

C. Precision, ROC

D. Recall, ROC

如下所示10个测试样本，‘Class’ 一栏表示每个测试样本的真正标签（P表示正例；N表示反例），‘Score’ 表示在某分类器中每个测试样本被预测为正样本的概率：

序号	1	2	3	4	5	6	7	8	9	10
Class	P	N	N	P	P	N	N	P	P	N
Score	0.93	0.85	0.80	0.7	0.55	0.50	0.40	0.3	0.2	0.1

请绘制 roc 曲线并计算 auc 指标

计算过程

阈值	TP	FP	TN	FN	TPR	FPR
∞	0	0	6	4	0.0	0.0
0.93	1	0	6	3	0.25	0.0
0.85	2	0	6	2	0.5	0.0
0.8	2	1	5	2	0.5	0.17
0.7	2	2	4	2	0.5	0.33
0.55	3	2	4	1	0.75	0.33
0.5	3	3	3	1	0.75	0.5
0.4	3	4	2	1	0.75	0.67
0.3	4	4	2	0	1.0	0.67
0.2	4	5	1	0	1.0	0.83
0.1	4	6	0	0	1.0	1.0



Q&A

TRANSWARP
星环科技